CESIZen

L'application de votre santé mentale

PLAN DE DÉPLOIEMENT, DE MAINTENANCE ET DE SÉCURISATION

Bloc 3 — Déployer et sécuriser les applications informatiques

Auteur	Barthélémy Le Bel
Formation	Concepteur Développeur d'Applications — Activité 3
Commanditaire	Ministère de la Santé et de la Prévention (simulation)
Version du document	1.0
Date	18 août 2026

Sommaire

Le sommaire se met à jour dans Word par un clic droit sur le champ, puis « Mettre à jour les champs » (ou touche F9).

1. Introduction et périmètre

1.1 Rappel du contexte

CESIZen est une plateforme grand public de prévention en santé mentale portée par le Ministère de la Santé et de la Prévention : contenus d'information, outils de diagnostic du stress et journal de suivi émotionnel. Deux caractéristiques déterminent l'ensemble des choix exposés ici.

- **L'application traite des données de santé**, sensibles au sens de l'article 9 du RGPD. Le journal d'émotions révèle l'état psychique de son auteur : une fuite ne serait pas un incident technique, mais une atteinte grave à la vie privée de personnes vulnérables.
- **L'application porte les couleurs de l'État**, ce qui en fait une cible de choix. L'hypothèse de travail n'est donc pas « si une attaque survient » mais « quand ».

Ces constats imposent un niveau d'exigence supérieur à celui d'une application grand public ordinaire, alors même que le budget reste contraint (75 000 €) et que l'équipe se réduit à un prestataire unique. Concilier ces deux réalités suppose de s'appuyer sur l'automatisation et sur des briques open source éprouvées, plutôt que sur des effectifs ou des licences que le projet ne peut pas financer.

1.2 Périmètre de l'activité 3

Ce dossier couvre exclusivement les compétences de l'activité 3. La conception (activité 1) et le développement (activité 2) sont acquis : le prototype fonctionnel existe, couvre les modules **Comptes utilisateurs**, **Informations** et **Tracker d'émotions**, et dispose de son cahier de tests. Ce document répond à trois questions :

- **Comment livrer** l'application de façon reproductible, tracée et réversible ? (ch. 2)
- **Comment la maintenir**, en traitant les anomalies dans les délais contractuels et en anticipant l'obsolescence technique ? (ch. 3)
- **Comment la protéger**, ainsi que ses données, et réagir lorsque la protection est mise en défaut ? (ch. 4)

1.3 Solution technique à déployer

Le dossier de l'activité 2 a retenu la pile suivante, qui conditionne les choix de déploiement et de sécurisation.

Couche	Technologie	Incidence sur le déploiement et la sécurité
Backend	Django 5.0 + Django REST Framework 3.15	ORM paramétré (protection native contre l'injection SQL), protections CSRF/XSS par défaut, `check --deploy` exploitable en intégration continue.
Base de données	PostgreSQL 16	Conformité ACID, `pgcrypto` pour le chiffrement au niveau colonne, sauvegarde logique par `pg_dump`.
Mobile	Expo React Native 51	Distribution par les magasins : le cycle de livraison mobile est dissocié de celui de l'API, avec une contrainte de rétrocompatibilité.
Authentification	JWT (djangorestframework-simplejwt)	Jetons de courte durée stockés dans le coffre sécurisé du terminal ; révocation par liste de refus.
Conteneurisation	Docker et Docker Compose	Environnements identiques du poste de développement à la production ; cloisonnement réseau et limitation des privilèges.
Forge logicielle	Git + GitHub (Actions, Issues, Projects)	Chaîne unique du code au ticket : versioning, tests, déploiement et suivi des incidents dans un seul outil.

Un choix structurant : une forge unique

Le sujet souhaite « un même outil capable de gérer le versioning, les tests unitaires et de non-régression ». GitHub couvre ces trois besoins et y ajoute le ticketing et le pilotage, garantissant une traçabilité continue : un ticket est lié à une branche, celle-ci à une *pull request*, puis à une exécution de tests et à un déploiement horodaté.

2. Plan de déploiement

2.1 Architecture cible

L'architecture retenue est conteneurisée et mono-serveur, placée derrière un reverse-proxy assurant la terminaison TLS. Elle est volontairement modeste : la charge attendue au démarrage du service ne justifie pas une orchestration Kubernetes, dont le coût d'exploitation absorberait une part du budget sans bénéfice mesurable. Le passage à l'échelle reste possible sans réécriture, les images produites étant identiques.

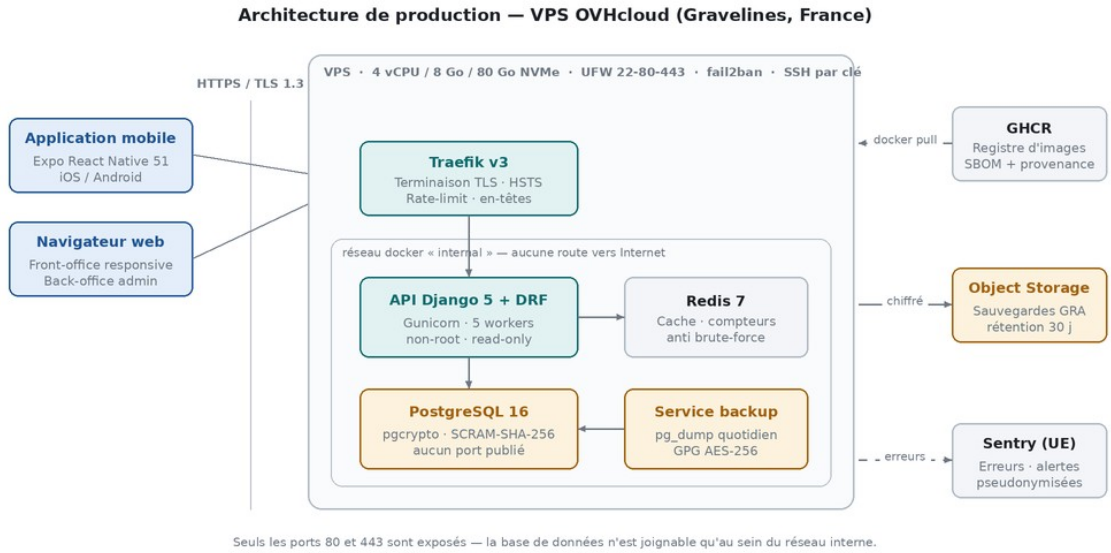


Figure 1 — Architecture de production. Les composants applicatifs sont isolés dans un réseau Docker interne sans route vers Internet.

Trois principes gouvernent ce schéma :

- **Une seule porte d'entrée.** Traefik est le seul conteneur exposant des ports publics (80 et 443). Le port 80 ne sert qu'à rediriger vers HTTPS et à répondre au défi de validation des certificats.
- **La base de données est inatteignable depuis Internet.** PostgreSQL et Redis sont placés sur un réseau Docker `internal`, sans route sortante : même en cas de compromission de l'API, l'attaquant n'obtient pas de canal d'exfiltration direct.
- **Le conteneur applicatif s'exécute sans privilèges.** Utilisateur non root, système de fichiers en lecture seule, toutes les capacités Linux retirées et `no-new-privileges` activé : l'exploitation d'une faille applicative ne se transforme pas mécaniquement en prise de contrôle de l'hôte.

2.2 Les trois environnements

Trois environnements sont définis et **intégralement configurés dans le dépôt** : composition Docker, procédure d'installation, chaîne de déploiement automatisée et contrôles associés. Le provisionnement effectif d'un serveur relève d'une décision d'exploitation, distincte de la configuration du déploiement : la bascule du mode de démonstration vers un déploiement réel tient à une variable de dépôt (DEPLOY_ENABLED) et à deux secrets SSH, sans modification du code ni des procédures.

	Développement	Test / Recette	Production
Finalité	Développer et déboguer	Valider fonctionnellement avec le client, exécuter les tests de non-régression	Servir les utilisateurs finaux
Hébergement	Poste du développeur	VPS OVHcloud — Gravelines (FR)	VPS OVHcloud — Gravelines (FR)
Dimensionnement	Docker Desktop / Compose	2 vCPU, 4 Go RAM, 40 Go SSD	4 vCPU, 8 Go RAM, 80 Go NVMe + Object Storage
Adresse	localhost:8000	recette.cesizen.fr	api.cesizen.fr
Données	Jeu de démonstration (`fixtures`), aucune donnée réelle	Extraction de production **anonymisée** (emails et commentaires remplacés)	Données réelles des utilisateurs

	Développement	Test / Recette	Production
Accès	Développeur uniquement	Restreint : filtrage par IP + authentification HTTP, noindex	Public
Déclencheur de déploiement	Manuel (`docker compose up`)	Automatique à chaque fusion sur `develop`	Publication d'une étiquette `vX.Y.Z` **+ approbation manuelle**
Sauvegardes	Aucune	Hebdomadaire	Quotidienne chiffrée, rétention 30 jours
Supervision	Journaux locaux	Sentry (projet distinct)	Sentry + supervision de disponibilité + alertes
Coût mensuel	0 €	≈ 12 €	≈ 35 € (VPS + stockage objet + nom de domaine)

Pourquoi les données de recette sont anonymisées

Recopier la base de production dans un environnement de test est la façon la plus courante de provoquer une violation de données : l'environnement de recette est moins surveillé, ses accès sont plus larges, et il finit par être oublié. Un script d'anonymisation (`manage.py anonymiser_extraction`) remplace les adresses électroniques, les prénoms et l'intégralité des commentaires du journal d'émotions avant tout import. Les volumes et les distributions statistiques sont conservés, afin que la recette reste représentative.

2.3 Gestion des sources et des versions

2.3.1 Outil retenu

Le versioning repose sur **Git**, avec **GitHub** comme dépôt distant. Ce choix n'est pas une évidence à réinterroger : Git est le standard de fait, son modèle distribué garantit qu'une copie complète de l'historique existe sur chaque poste, et GitHub apporte les fonctions manquantes pour un projet public — protection de branches, revue de code, détection de secrets, avis de sécurité privés.

2.3.2 Modèle de branches

Un modèle GitFlow allégé est appliqué, adapté à une équipe réduite : les branches `release` de GitFlow sont supprimées, car elles n'ont d'intérêt qu'avec plusieurs équipes travaillant en parallèle.

Branche	Rôle	Protection	Environnement
`main`	Reflète exactement ce qui tourne en production ; chaque état livré y est étiqueté	Fusion par *pull request* uniquement, CI verte obligatoire, historique linéaire, `force-push` interdit	Production
`develop`	Branche d'intégration : tout ce qui est validé et attend une livraison	Fusion par *pull request*, CI verte obligatoire	Recette
`feature/<n°ticket>-<intitulé>`	Développement d'une évolution	Aucune	Local
`fix/<n°ticket>-<intitulé>`	Correction d'anomalie non urgente	Aucune	Local
`hotfix/<n°ticket>-<intitulé>`	Correction urgente, branchée sur `main` et fusionnée dans `main` puis `develop`	Revue accélérée, CI verte obligatoire	Production

2.3.3 Conventions et traçabilité

- **Messages de commit normalisés** (*Conventional Commits*) : `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`, `security:`. Cette normalisation n'est pas cosmétique : elle permet de générer automatiquement le journal des modifications et de déduire le numéro de version à publier.
- **Versionnement sémantique** (`MAJEUR.MINEUR.CORRECTIF`) : le numéro majeur signale une rupture de compatibilité de l'API — information critique pour l'application mobile, dont les versions anciennes restent installées sur les terminaux pendant des mois après une mise à jour.

- **Chaque commit référence un ticket.** Un développement sans ticket associé est refusé en revue : c'est la condition pour qu'un audit puisse remonter d'une ligne de code jusqu'à la demande qui l'a motivée.
- **Journal des modifications** (`CHANGELOG.md`) tenu au format *Keep a Changelog* ; son absence pour une version bloque la mise en production (contrôle automatisé).

2.3.4 Versioning de la documentation

La documentation technique — guide d'installation, schémas d'architecture, procédures d'exploitation, décisions d'architecture — est versionnée **dans le même dépôt** que le code, au format Markdown, sous `docs/`. Une documentation stockée dans un espace bureautique séparé se désynchronise systématiquement du code ; hébergée dans le dépôt, elle est modifiée par la même *pull request* que le changement qu'elle décrit, et la revue de code porte aussi sur elle.

2.4 Automatisation : intégration et déploiement continus

2.4.1 Outil retenu et justification

L'automatisation repose sur **GitHub Actions**. Trois solutions ont été comparées.

Critère	GitHub Actions	GitLab CI	Jenkins
Intégration au dépôt	Native (dépôt déjà sur GitHub)	Native, mais impose de migrer le dépôt	Externe : connecteur et jetons à maintenir
Coût pour le projet	Gratuit (dépôt public), 2 000 min/mois en privé	Gratuit jusqu'à 400 min/mois	Gratuit, mais nécessite un serveur dédié (≈ 10 €/mois + exploitation)
Charge d'exploitation	Nulle : exécuteurs hébergés	Nulle en SaaS	Élevée : mises à jour, greffons, sécurisation du serveur
Sécurité de la chaîne	Environnements protégés, `GITHUB_TOKEN` à portée réduite, détection de secrets	Équivalent	À construire entièrement
Verdict	**Retenu**	Écarté : bénéfice nul au prix d'une migration	Écarté : le serveur d'intégration deviendrait lui-même une surface d'attaque à maintenir

Le facteur décisif est la charge d'exploitation. Avec un prestataire unique, chaque serveur supplémentaire à administrer est du temps soustrait à la maintenance de l'application elle-même — et une surface d'attaque de plus.

2.4.2 Chaîne de livraison

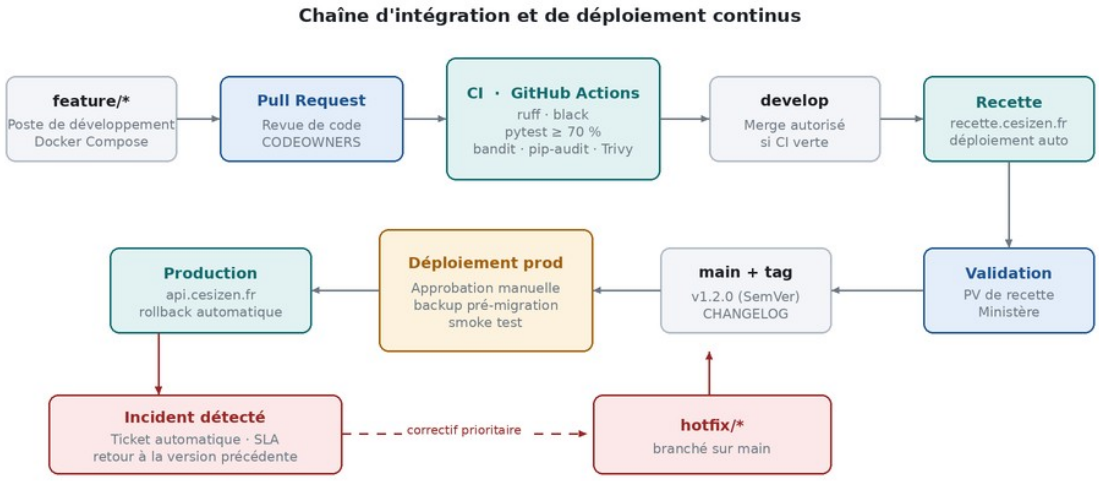


Figure 2 — De la branche de développement à la production, avec traitement des incidents.

Le processus d'intégration continue s'exécute à chaque *pull request* et bloque la fusion en cas d'échec. Il est décomposé en quatre travaux, le premier échouant vite pour ne pas consommer inutilement du temps de calcul.

Travail	Contenu	Condition de blocage
Qualité de code	`ruff` (lint et ordre des imports), `black --check` (formatage)	Toute violation
Tests	Contrôle des migrations manquantes, `pytest` sur une base PostgreSQL réelle (service éphémère), mesure de couverture	Un test en échec ou une couverture inférieure à **70 %**
Sécurité	`bandit` (analyse statique Python), `pip-audit` (vulnérabilités connues des dépendances), `manage.py check --deploy`, `Trivy` (image Docker)	Vulnérabilité **haute ou critique**, ou avertissement de configuration Django
Mobile	`eslint`, tests Jest, `npm audit`	Test en échec ou vulnérabilité haute

Le déploiement continu se décline différemment selon la cible, et c'est délibéré :

- **Recette : entièrement automatique.** Toute fusion sur `develop` est déployée sans intervention. L'objectif est que le Ministère dispose en permanence d'un environnement à jour pour sa recette, sans avoir à demander une livraison.
- **Production : automatisée mais déclenchée et approuvée par un humain.** La publication d'une étiquette `vX.Y.Z` lance le processus, qui se met en pause sur un environnement GitHub protégé jusqu'à approbation explicite. Le déploiement continu intégral vers un service public traitant des données de santé serait un choix difficile à défendre : la décision de livrer reste un acte de responsabilité.

2.4.3 Mode de démonstration

Les travaux agissant sur une infrastructure réelle sont conditionnés à la variable de dépôt `DEPLOY_ENABLED`. En son absence, un travail de démonstration s'exécute à leur place : il valide la composition de production, vérifie que le réseau interne reste cloisonné, qu'aucun port PostgreSQL n'est publié et que le durcissement du conteneur applicatif est intact, puis déroule la séquence de déploiement qui serait exécutée. Ce dispositif évite qu'un dépôt non relié à un serveur affiche des exécutions en échec, tout en conservant des contrôles réels et vérifiables.

2.4.4 Garanties de réversibilité

Un plan de déploiement ne vaut que par sa capacité à revenir en arrière. Quatre mécanismes sont en place :

Mécanisme	Fonctionnement	Objectif
Images étiquetées et immuables	Chaque image porte le SHA du commit ou l'étiquette de version ; aucune image n'est réécrite	Revenir à une version antérieure revient à changer une variable et à relancer le service
Sauvegarde avant migration	Un `pg_dump` est exécuté automatiquement avant toute migration de schéma	Restaurer l'état exact précédant une migration destructrice
Vérification post-déploiement	Interrogation répétée du point de contrôle `/api/health/` pendant deux minutes	Détecter l'échec avant les utilisateurs
Retour arrière automatique	En cas d'échec de la vérification, la version précédente est redéployée et un ticket d'incident est ouvert automatiquement	Ramener la production à un état sain sans attendre une intervention humaine

2.5 Procédure de déploiement pas à pas

Les étapes ci-dessous sont celles réellement exécutées. Les étapes automatisées sont identifiées ; les autres relèvent d'une intervention humaine.

2.5.1 Installation initiale d'un serveur (opération unique)

#	Étape	Commande ou mécanisme	Durée	Responsable
1	Commande du VPS et du nom de domaine	Console OVHcloud, région Gravelines	30 min	Prestataire
2	Durcissement du système	Compte `deploy` sans mot de passe, SSH par clé uniquement, connexion `root` interdite	20 min	Prestataire
3	Pare-feu et anti-bourrage	`ufw` : seuls 22, 80 et 443 ouverts ; `fail2ban` activé	10 min	Prestataire
4	Correctifs de sécurité automatiques	`unattended-upgrades`	5 min	Prestataire

#	Étape	Commande ou mécanisme	Durée	Responsable
5	Installation de Docker	Dépôt officiel, `docker compose` v2	10 min	Prestataire
6	Récupération du dépôt et des secrets	`git clone` puis dépôt du fichier `.env` en `chmod 600`	15 min	Prestataire
7	Démarrage de la pile	`docker compose -f docker-compose.prod.yml up -d`	5 min	Prestataire
8	Migrations et jeu de référence	`manage.py migrate` puis chargement du référentiel d'émotions	5 min	Prestataire
9	Vérification du certificat et des en-têtes	Contrôle TLS externe, cible : note A ou supérieure	15 min	Prestataire
10	Enregistrement des secrets de déploiement	Clé SSH et hôte déclarés dans les secrets GitHub	10 min	Prestataire

Durée totale : environ **2 heures**. Cette procédure n'est rejouée qu'en cas de reconstruction complète du serveur, ce qui constitue également le plan de reprise d'activité en cas de perte du VPS.

2.5.2 Déploiement en recette (automatisé)

#	Étape	Déclenchement	Automatisé
1	Fusion de la *pull request* dans `develop`	Action du développeur après revue	—
2	Construction de l'image et publication au registre GHCR	Processus `Déploiement Recette`	Oui
3	Sauvegarde de la base de recette	Connexion SSH depuis le processus	Oui
4	Récupération de la nouvelle image	`docker compose pull`	Oui
5	Application des migrations	`manage.py migrate --noinput`	Oui
6	Bascule du conteneur applicatif	`docker compose up -d --no-deps backend`	Oui
7	Vérification de disponibilité	10 interrogations de `/api/health/`	Oui
8	Notification en cas d'échec	Message sur le canal projet	Oui

Durée : **4 à 6 minutes** sans intervention humaine.

2.5.3 Mise en production (automatisée sous approbation)

#	Étape	Déclenchement	Automatisé
1	Recette validée par le Ministère (procès-verbal signé)	Réunion de recette	—
2	Fusion de `develop` dans `main`	*Pull request* de livraison	—
3	Rédaction du journal des modifications et pose de l'étiquette `vX.Y.Z`	`git tag -a` puis `git push --tags`	—
4	Contrôles préalables : l'étiquette pointe bien sur `main`, le journal des modifications comporte la version	Processus `Déploiement Production`	Oui
5	Construction de l'image avec attestation de provenance et inventaire logiciel (SBOM)	Processus	Oui
6	**Attente d'approbation manuelle**	Environnement GitHub protégé	Non — validation explicite
7	Sauvegarde de la base et mémorisation de la version en service	Connexion SSH	Oui
8	Migrations, bascule applicative, vérification de disponibilité	Connexion SSH	Oui
9	Si échec : retour à la version précédente et ouverture d'un ticket d'incident	Processus	Oui
10	Publication de la mise à jour mobile si nécessaire	`eas submit` (Expo)	Partiel
11	Communication au Ministère	Note de livraison	—

Durée : **8 à 12 minutes**, hors délai d'approbation et hors validation par les magasins d'applications (compter 24 à 48 heures pour ces derniers, ce qui impose de conserver la compatibilité ascendante de l'API pendant cette fenêtre).

2.6 Ressources nécessaires

2.6.1 Ressources humaines

Rôle	Charge	Responsabilités
Développeur / exploitant (prestataire)	0,2 ETP en phase de maintenance	Développement, exécution des livraisons, traitement des tickets, veille
Référent métier (Ministère)	2 jours par livraison	Recette fonctionnelle, signature du procès-verbal, arbitrage des priorités
Délégué à la protection des données (Ministère)	Ponctuel	Validation de l'analyse d'impact, avis sur les évolutions touchant aux données personnelles, notification à la CNIL
Astreinte technique	Heures ouvrées	Prise en compte des incidents bloquants dans le délai contractuel

2.6.2 Ressources techniques et budgétaires

Poste	Détail	Coût annuel
VPS de production	OVHcloud, 4 vCPU / 8 Go / 80 Go NVMe, Gravelines	≈ 300 €
VPS de recette	OVHcloud, 2 vCPU / 4 Go / 40 Go	≈ 140 €
Stockage objet (sauvegardes)	50 Go, région GRA	≈ 30 €
Nom de domaine et certificats	`cesizen.fr` ; TLS par Let's Encrypt (gratuit)	≈ 15 €
Forge logicielle	GitHub — Actions, Issues, Projects, avis de sécurité	0 €
Supervision des erreurs	Sentry, offre gratuite (instance UE)	0 €
Comptes développeur mobile	Apple Developer (99 $) + Google Play (25 $ une fois)	≈ 115 €
Total infrastructure		**≈ 600 € / an**

Ce budget représente moins de 1 % de l'enveloppe du projet. Ce n'est pas un hasard : l'ensemble de la chaîne repose sur des composants libres et sur des services dont l'offre gratuite suffit à l'usage. L'essentiel du coût de possession est du temps humain, ce qui renforce l'intérêt de l'automatisation décrite en 2.4.

2.7 Pilotage du déploiement

Le pilotage s'appuie sur les quatre indicateurs DORA, mesurés à partir de l'historique des exécutions GitHub Actions et complétés par un indicateur de sécurité.

Indicateur	Définition	Cible	Source
Fréquence de livraison	Nombre de mises en production par mois	⩾ 1	Étiquettes Git
Délai de mise à disposition	Temps entre la fusion et l'exécution en production	< 5 jours	Historique des exécutions
Taux d'échec des changements	Part des livraisons suivies d'un retour arrière ou d'un correctif urgent	< 15 %	Tickets `incident`
Délai moyen de rétablissement	Temps entre la détection d'un incident bloquant et le retour au service nominal	< 3 h ouvrées	Horodatage des tickets
Couverture de tests	Part du code couverte par les tests automatisés	⩾ 70 %	Rapport `pytest-cov`
Vulnérabilités ouvertes	Nombre de vulnérabilités hautes ou critiques non traitées	0	Onglet *Security*

Ces indicateurs sont revus mensuellement en comité de suivi. Ils ne servent pas seulement à rendre compte : une dégradation du délai de rétablissement signale généralement une insuffisance de tests ou de supervision, bien avant qu'un incident majeur ne survienne.

3. Plan de maintenance

3.1 Outil de gestion des corrections et évolutions

3.1.1 Solution retenue

La gestion des demandes repose sur **GitHub Issues** pour les tickets et **GitHub Projects** pour le pilotage. Trois solutions ont été examinées.

Critère	GitHub Issues + Projects	Jira	Redmine
Lien avec le code	Direct : un ticket se ferme automatiquement à la fusion de la *pull request* qui le référence	Par connecteur à configurer	Par connecteur
Coût	Inclus	≈ 8 €/utilisateur/mois	Gratuit, mais serveur à héberger et maintenir
Formulaires structurés	Oui, au format YAML versionné dans le dépôt	Oui, configurés dans l'interface	Limités
Automatisations	Actions GitHub : création de tickets, étiquetage, transitions	Règles d'automatisation intégrées	Greffons
Adapté à l'équipe	Oui : un prestataire, un dépôt, un outil	Surdimensionné pour ce projet	Charge d'exploitation injustifiée
Verdict	**Retenu**	Écarté	Écarté

Le critère déterminant est le lien natif entre le ticket et le code. Lorsque la correction d'une anomalie ferme automatiquement son ticket et que l'exécution de déploiement y est rattachée, la traçabilité exigée par le Ministère est obtenue sans saisie manuelle — donc sans risque d'oubli.

3.1.2 Configuration mise en place

La configuration est versionnée dans le dépôt, sous `.github/` :

Élément	Contenu	Objectif
Trois formulaires de saisie	Anomalie, évolution, incident de sécurité	Imposer les informations indispensables dès l'ouverture : la sévérité, l'environnement et les étapes de reproduction ne sont plus à réclamer
Référentiel d'étiquettes	Nature, sévérité, priorité, module fonctionnel, cycle de vie	Filtrer le portefeuille sans outil supplémentaire ; la sévérité détermine directement le délai contractuel applicable
Tableau de pilotage (*Project*)	Colonnes : À qualifier → Qualifié → En cours → En revue → En recette → Livré ; champs Sévérité, Échéance, Charge estimée	Vue unique du portefeuille, filtrable par module ou par échéance
Modèle de *pull request*	Liste de contrôle : tests, couverture, absence de secret, réversibilité de la migration, impact sur les données personnelles	Transformer les bonnes pratiques en gestes systématiques plutôt qu'en intentions
Création automatique de tickets	Échec de mise en production, audit hebdomadaire de sécurité	Aucun signal automatique ne se perd : il devient un élément du portefeuille

3.2 Cycle de vie d'un ticket

Étape	Acteur	Contenu	Délai
1. Ouverture	Ministère, utilisateur, supervision ou automatisme	Saisie du formulaire ; le ticket arrive dans la colonne *À qualifier*	—
2. Qualification	Prestataire	Reproduction du problème, confirmation ou révision de la sévérité, affectation d'un module, estimation de la charge	Selon le délai de prise en compte (§ 3.3)
3. Arbitrage	Prestataire + Ministère	Priorisation ; pour une évolution, chiffrage et validation préalable	Comité hebdomadaire
4. Réalisation	Prestataire	Branche dédiée, correction accompagnée d'un test reproduisant l'anomalie	Selon le délai de correction
5. Revue et	Prestataire	*Pull request*, intégration continue verte, revue selon	—

Étape	Acteur	Contenu	Délai
intégration		la liste de contrôle	
6. Recette	Ministère	Vérification sur `recette.cesizen.fr`	—
7. Livraison	Prestataire	Étiquetage, mise en production, clôture automatique du ticket	—
8. Clôture	Ministère	Confirmation de la correction en production	—

Une règle systématique : pas de correction sans test

Toute correction d'anomalie est accompagnée d'un test automatisé qui échoue avant le correctif et réussit après. Ce test rejoint la suite exécutée à chaque *pull request* : l'anomalie corrigée devient un cas de non-régression permanent. C'est le mécanisme par lequel la couverture de tests progresse au rythme de la vie du produit, sans campagne d'écriture de tests dédiée que le budget ne permettrait pas.

3.3 Maintenance corrective

3.3.1 Qualification de la sévérité et engagements de délai

La sévérité est déterminée par l'impact sur le service rendu, et non par la difficulté technique de la correction. Elle est saisie par le demandeur puis confirmée — ou révisée, avec justification écrite dans le ticket — lors de la qualification.

Sévérité	Définition	Prise en compte et diagnostic	Correction	Exemple CESIZen
Bloquant — priorité critique	Interruption non planifiée rendant le service impossible, sans contournement	1 heure ouvrée	3 heures ouvrées	API inaccessible ; impossibilité de s'authentifier
Bloquant — priorité forte	Interruption sans contournement sur un périmètre restreint	2 heures ouvrées	6 heures ouvrées	Enregistrement d'une émotion impossible sur Android
Majeur	Service inopérant par intermittence ou par recours à un contournement	7 heures ouvrées	16 heures ouvrées	Le courriel de réinitialisation du mot de passe n'arrive qu'une fois sur deux
Mineur (traité par lots)	Simple altération de la qualité de service, le service restant opérationnel	1 jour ouvré	40 heures ouvrées	Libellé erroné ; graphique mal cadré sur petit écran

Ces délais sont ceux imposés par le Ministère. Ils sont mesurés à partir de l'horodatage d'ouverture du ticket, information non modifiable, et le respect du délai est un indicateur du tableau de bord de pilotage (§ 3.6).

3.3.2 Procédure de traitement d'un incident bloquant

#	Action	Objectif
1	Accusé de réception dans le ticket et information du référent Ministère	Le client sait que l'incident est pris en charge et par qui
2	Constat de l'impact : services touchés, nombre d'utilisateurs, données concernées	Décider si l'incident relève aussi du processus de sécurité (§ 4.6)
3	Recherche d'un contournement immédiat : retour à la version précédente, désactivation ciblée d'une fonctionnalité, page de maintenance	Rétablir un service, même dégradé, avant de chercher la cause profonde
4	Diagnostic à partir des journaux Sentry, Traefik et applicatifs	Identifier la cause exacte plutôt que d'en traiter le symptôme
5	Correction sur une branche `hotfix/`, accompagnée du test de non-régression	Éviter la réapparition de l'anomalie
6	Livraison accélérée : intégration continue complète, mais approbation en procédure d'urgence	Ne pas sacrifier les contrôles automatisés sous la pression du délai
7	Vérification en production et clôture avec le référent	Confirmer le rétablissement auprès de celui qui a signalé l'incident
8	Retour d'expérience si l'incident a dépassé le délai contractuel	Alimenter le plan d'action : supervision manquante, test absent, procédure à corriger

3.4 Maintenance évolutive

Chaque demande d'évolution suit un circuit distinct de celui des anomalies : une évolution n'est jamais réalisée sans validation préalable du commanditaire, puisqu'elle engage un coût.

Étape	Production attendue	Délai
Expression du besoin	Ticket au format `[EVOL]` : besoin, justification, priorité MoSCoW, critères d'acceptation	—
Analyse d'opportunité	Avis du prestataire : faisabilité, impacts sur la sécurité et les données personnelles, alternatives	5 jours ouvrés
Proposition chiffrée	Documentation fonctionnelle et technique, charge estimée, délai, coût, incidences sur la maintenance	10 jours ouvrés
Validation	Bon pour accord du Ministère ; sans réponse, le ticket passe en *En attente client* et sort du portefeuille actif	—
Planification	Rattachement à une version cible et à un jalon du tableau de pilotage	—
Réalisation et recette	Développement, tests, recette, procès-verbal	Selon le chiffrage

La priorisation utilise la méthode **MoSCoW** (*Must*, *Should*, *Could*, *Won't*), arbitrée en comité mensuel. Les critères de pondération retenus sont, par ordre décroissant : conformité réglementaire, sécurité, nombre d'utilisateurs concernés, effort de réalisation, dette technique induite.

3.5 Veille technologique et pérennité

La pérennité de CESIZen dépend moins de la qualité initiale du code que de la capacité à suivre l'évolution de ses dépendances. Une application Django figée devient vulnérable en dix-huit mois, sans qu'une seule ligne de son code n'ait changé. La veille est donc traitée comme une activité planifiée, et non comme une bonne intention.

3.5.1 Sources suivies

Domaine	Sources	Rythme
Sécurité applicative	Bulletins de sécurité Django, avis GitHub (GHSA), base CVE/NVD, alertes CERT-FR (ANSSI)	Quotidien (notifications)
Dépendances	Dependabot, `pip-audit`, `npm audit`, Trivy sur l'image de production	Hebdomadaire (automatisé)
Écosystème Python et Django	Notes de version Django, calendrier de fin de support, *Django News*	Mensuel
Écosystème mobile	Notes de version Expo et React Native, exigences Apple et Google en matière de version cible du SDK	Mensuel
Conformité	Recommandations CNIL (santé, durées de conservation), référentiels et guides d'hygiène de l'ANSSI	Trimestriel
Pratiques de sécurité	OWASP Top 10, OWASP Mobile Top 10, ASVS	Semestriel

3.5.2 Automatisation de la veille

Deux mécanismes garantissent que la veille produit des actions et non des rapports inconsultés :

- **Dependabot** ouvre chaque semaine des *pull requests* de montée de version. Les versions mineures et correctives sont regroupées afin de limiter le nombre de demandes à traiter ; les versions majeures restent isolées, car elles nécessitent un arbitrage réel. Chaque *pull request* passe l'intégration continue complète : si les tests restent verts, la montée de version se fusionne en quelques minutes.
- **Un processus planifié hebdomadaire** exécute `pip-audit`, un scan Trivy de l'image de production et une analyse CodeQL, puis **crée automatiquement un ticket** portant les résultats. Ce ticket doit être clos chaque semaine, même lorsqu'il ne relève aucune vulnérabilité : la trace de la veille est ainsi produite par le processus lui-même.

3.5.3 Critères de décision

Toute veille finit par poser la question de l'adoption. Les règles retenues sont explicites, afin d'éviter les décisions au cas par cas :

Situation	Décision	Délai
Vulnérabilité critique ou haute exploitable à distance	Correctif immédiat, procédure d'incident, livraison en urgence	72 heures
Vulnérabilité moyenne ou faible	Intégration au lot de maintenance mensuel	1 mois
Version corrective (`5.0.x`)	Adoption automatique si l'intégration continue reste verte	Hebdomadaire
Version mineure (`5.x`)	Adoption après lecture des notes de version et recette	Trimestriel
Version majeure (`6.0`)	Étude d'impact chiffrée, traitée comme une évolution	Sur validation
Fin de support d'une version (LTS)	Migration planifiée **6 mois avant** la date de fin	Anticipé
Dépendance abandonnée par son mainteneur	Recherche d'alternative, ticket de dette technique	Trimestriel

La règle des six mois avant fin de support est déterminante : elle évite de mener une montée de version majeure dans l'urgence, au moment précis où une faille non corrigée est publiée sur une version devenue non maintenue.

3.6 Tableau de bord de pilotage de la maintenance

Le tableau de bord est construit dans GitHub Projects à partir des champs des tickets, et présenté au comité mensuel.

Indicateur	Calcul	Cible
Respect du délai de prise en compte	Part des tickets qualifiés dans le délai contractuel	100 %
Respect du délai de correction	Part des anomalies corrigées dans le délai contractuel	⩾ 95 %
Portefeuille ouvert	Nombre d'anomalies non résolues, par sévérité	0 bloquant, < 5 majeurs
Âge moyen des tickets ouverts	Moyenne des durées depuis l'ouverture	< 30 jours
Taux de réouverture	Part des tickets rouverts après clôture	< 5 %
Dette de sécurité	Vulnérabilités hautes ou critiques ouvertes	0
Fraîcheur des dépendances	Part des dépendances à jour à une version mineure près	⩾ 90 %
Traçabilité de la veille	Tickets de veille hebdomadaires traités	100 %

4. Plan de sécurisation

4.1 Méthode d'analyse des risques

L'analyse s'inspire de la méthode **EBIOS Risk Manager** de l'ANSSI, ramenée à une échelle proportionnée au projet. Chaque risque est évalué selon deux axes, puis quantifié par une criticité :

> **Criticité = Vraisemblance × Impact**
>
> **Vraisemblance (V)** — 1 : rare (aucun cas connu sur un service comparable) · 2 : possible (déjà observé dans le secteur) · 3 : probable (attendu au moins une fois par an) · 4 : quasi certain (tentatives permanentes).
>
> **Impact (I)** — 1 : mineur (gêne, sans perte) · 2 : modéré (dégradation temporaire) · 3 : majeur (interruption de service ou perte de données limitée) · 4 : critique (violation de données de santé, atteinte à l'image de l'État, sanction réglementaire).
>
> **Niveaux de criticité** — 1 à 3 : faible (surveiller) · 4 à 7 : moyen (traiter dans le plan de maintenance) · 8 à 11 : élevé (traiter en priorité) · 12 à 16 : critique (traitement immédiat, aucune mise en production sans mesure).

L'échelle d'impact est volontairement sévère : la nature des données traitées fait qu'une fuite, même de faible volume, atteint directement des personnes en situation de fragilité psychique. Aucun risque touchant à la confidentialité de ces données ne peut donc être classé en dessous du niveau majeur.

4.2 Matrice des risques

ID	Risque identifié	Catégorie	V	I	C	Niveau
R01	Injection SQL ou NoSQL via les paramètres de l'API	Applicatif	2	4	8	Élevé
R02	Injection de script (XSS) stocké via l'éditeur d'articles du back-office	Applicatif	3	3	9	Élevé
R03	Compromission de comptes par bourrage d'identifiants ou attaque par force brute	Authentification	3	4	12	Critique
R04	Vol ou rejeu d'un jeton JWT (terminal compromis, interception)	Authentification	2	4	8	Élevé
R05	Accès aux données d'un autre utilisateur (référence directe non contrôlée)	Contrôle d'accès	2	4	8	Élevé
R06	Exposition d'un secret dans le dépôt ou les journaux (clé, mot de passe)	Chaîne de développement	3	4	12	Critique
R07	Exploitation d'une dépendance vulnérable connue	Chaîne d'approvisionnement	4	3	12	Critique
R08	Déni de service : saturation de l'API ou de la base	Disponibilité	3	3	9	Élevé
R09	Perte ou corruption de données (erreur de migration, suppression)	Données	2	4	8	Élevé
R10	Indisponibilité du serveur ou du centre de données	Disponibilité	2	3	6	Moyen
R11	Exfiltration de la base de données (données de santé)	Confidentialité	2	4	8	Élevé
R12	Erreur humaine lors d'une mise en production	Exploitation	3	3	9	Élevé
R13	Usage abusif des droits d'administration (malveillance ou négligence interne)	Contrôle d'accès	1	4	4	Moyen
R14	Non-conformité RGPD : conservation excessive, consentement non recueilli	Conformité	2	4	8	Élevé
R15	Compromission de la chaîne d'intégration continue (jeton, action tierce)	Chaîne de développement	2	4	8	Élevé
R16	Hameçonnage ciblant un compte d'administration	Authentification	3	3	9	Élevé

Répartition : 3 risques critiques, 8 élevés, 5 moyens. Les trois risques critiques — bourrage d'identifiants (R03), exposition de secrets (R06) et dépendance vulnérable (R07) — partagent une caractéristique : leur vraisemblance est élevée parce qu'ils ne supposent aucun ciblage particulier. Ce sont des attaques opportunistes, menées en masse et automatiquement, qui touchent tout service exposé sur Internet. Ils sont donc traités par des mesures permanentes et automatisées, et non par de la vigilance.

4.3 Actions préventives et correctives

Les seize risques identifiés font chacun l'objet de mesures préventives — destinées à réduire la vraisemblance ou l'impact — et de mesures correctives, appliquées lorsque le risque se matérialise. La colonne « Criticité résiduelle » indique le niveau atteint après application des mesures préventives.

ID	Actions préventives	Actions correctives	Criticité résiduelle
R01	ORM Django exclusivement (requêtes paramétrées), aucune requête SQL construite par concaténation ; validation stricte par les *serializers* DRF ; analyse `bandit` et CodeQL à chaque *pull request*	Correctif d'urgence ; recherche d'exploitation dans les journaux ; rotation des identifiants de base ; procédure d'incident N2 minimum	Faible
R02	Assainissement du HTML par liste blanche (`bleach`) au stockage et à l'affichage ; en-tête *Content-Security-Policy* restrictif ; échappement automatique des gabarits Django	Purge des contenus infectés en base ; invalidation des sessions concernées ; analyse des comptes ayant consulté la page piégée	Faible
R03	Hachage Argon2 des mots de passe ; politique de complexité et contrôle contre les listes de mots de passe compromis ; limitation à 5 tentatives par minute et par IP (Traefik + compteurs Redis) ; **authentification à deux facteurs obligatoire pour les comptes d'administration**	Blocage des adresses sources ; invalidation globale des jetons ; réinitialisation forcée des comptes touchés ; information des utilisateurs concernés	Moyen
R04	Jeton d'accès de courte durée (24 h) et jeton de rafraîchissement rotatif avec liste de refus ; stockage dans le coffre sécurisé du terminal (`SecureStore` / Keychain) et jamais en stockage local ; HTTPS strict avec HSTS	Révocation immédiate du jeton et de la famille de rafraîchissement ; déconnexion de toutes les sessions de l'utilisateur	Faible
R05	Filtrage systématique des jeux de données sur `request.user` ; permissions DRF au niveau de l'objet ; **tests automatisés d'autorisation** vérifiant qu'un utilisateur ne peut accéder à la ressource d'un autre ; identifiants UUID	Correctif immédiat ; reconstitution des accès indus à partir des journaux ; qualification en violation de données si des accès sont avérés	Faible
R06	Fichiers `.env` exclus du suivi Git ; détection de secrets et blocage à la poussée activés sur le dépôt ; `gitleaks` en intégration continue ; secrets stockés dans GitHub Secrets ; journaux applicatifs expurgés des données sensibles	Révocation et rotation immédiates du secret exposé ; réécriture de l'historique si nécessaire ; recherche d'utilisation frauduleuse	Moyen
R07	Dependabot hebdomadaire ; `pip-audit`, `npm audit` et Trivy bloquants en intégration continue au niveau haut ou critique ; inventaire logiciel (SBOM) produit à chaque livraison ; images de base minimales (`slim`, `alpine`)	Application du correctif sous 72 h ; à défaut de correctif disponible, contournement ou désactivation de la fonctionnalité concernée	Moyen
R08	Limitation de débit au niveau du proxy (100 req/s par IP, 5/min sur l'authentification) ; protection anti-DDoS de l'hébergeur ; délais d'attente et recyclage des processus Gunicorn ; pagination obligatoire des collections	Durcissement temporaire des seuils ; blocage des sources ; page de maintenance ; escalade auprès de l'hébergeur	Moyen
R09	Sauvegarde quotidienne chiffrée avec vérification d'intégrité automatique ; **sauvegarde systématique avant toute migration** ; migrations réversibles exigées en revue ; test de restauration trimestriel documenté	Restauration à partir de la sauvegarde valide la plus récente ; analyse de l'écart de données ; information du Ministère	Faible
R10	Supervision de disponibilité avec alerte ; infrastructure entièrement décrite dans le dépôt, permettant de reconstruire un serveur en moins de 2	Reconstruction sur un nouveau VPS puis restauration ; objectifs : reprise en 4 h, perte de données maximale 24 h	Moyen

ID	Actions préventives	Actions correctives	Criticité résiduelle
	h ; instantanés hebdomadaires du VPS		
R11	Base inaccessible depuis Internet (réseau Docker interne) ; chiffrement au niveau colonne des commentaires du journal d'émotions ; principe de moindre privilège (rôle en lecture seule distinct pour les statistiques) ; sauvegardes chiffrées AES-256	Procédure de crise N3 : confinement, notification à la CNIL sous 72 h, information des personnes concernées, rotation de tous les secrets	Moyen
R12	Intégration continue bloquante ; environnement de production protégé par approbation manuelle ; sauvegarde préalable et vérification post-déploiement ; retour arrière automatique	Retour à la version précédente (automatique) ; ticket d'incident créé automatiquement ; retour d'expérience	Faible
R13	Journalisation horodatée et inaltérable des actions d'administration ; authentification à deux facteurs ; revue trimestrielle des comptes et des droits ; suppression des accès dès la fin d'une mission	Suspension du compte ; analyse des journaux d'administration ; qualification en violation de données le cas échéant	Faible
R14	Registre des traitements tenu à jour ; analyse d'impact réalisée et révisée ; purge automatique aux échéances de conservation ; contrôle « impact sur les données personnelles » dans chaque *pull request*	Régularisation immédiate ; information du délégué à la protection des données ; documentation de l'écart et de sa correction	Faible
R15	Permissions minimales accordées au jeton d'intégration ; actions tierces épinglées à une version vérifiée ; environnements protégés avec approbation ; aucun secret de production accessible depuis un processus déclenché par une *pull request* externe	Rotation de tous les secrets ; audit des exécutions récentes ; vérification de l'intégrité des images publiées	Faible
R16	Authentification à deux facteurs obligatoire ; sensibilisation des administrateurs ; aucune demande d'identifiants par courriel ; domaine protégé par les enregistrements SPF, DKIM et DMARC	Réinitialisation des accès ; analyse des connexions récentes ; signalement de la campagne au Ministère	Moyen

Après application des mesures préventives, **aucun risque ne demeure au niveau critique ni élevé** : 10 risques sont ramenés au niveau faible et 6 au niveau moyen. Les risques résiduels moyens correspondent à des menaces que l'on peut réduire mais non supprimer — un service exposé sur Internet subira toujours des tentatives d'intrusion et des campagnes d'hameçonnage. Ces risques sont acceptés, documentés, et surveillés par les indicateurs du § 3.6.

4.4 Solutions de chiffrement

Le chiffrement est appliqué à quatre niveaux, chacun répondant à une menace distincte. Les présenter séparément évite le raccourci fréquent selon lequel « le site est en HTTPS, donc les données sont chiffrées ».

Niveau	Mise en œuvre	Menace couverte
En transit	TLS 1.3 (1.2 minimum) via Traefik, certificats Let's Encrypt renouvelés automatiquement ; redirection HTTP→HTTPS systématique ; HSTS un an avec `preload` ; épinglage de certificat côté application mobile	Interception réseau, attaque de l'intercepteur sur un réseau Wi-Fi public
Mots de passe	Hachage **Argon2id** (lauréat du concours *Password Hashing Competition*, résistant aux attaques par matériel dédié), sel unique par utilisateur, paramètres de coût révisés annuellement	Réutilisation des mots de passe après compromission de la base
Données sensibles en base	Chiffrement au niveau colonne (Fernet AES-128-CBC + HMAC, clé applicative distincte de la base) des **commentaires libres du journal d'émotions**, seul champ en texte libre susceptible de contenir un récit personnel identifiant	Lecture directe de la base ou d'une sauvegarde par un tiers
Sauvegardes	Chiffrement GPG symétrique AES-256 avant tout transfert ; la phrase secrète n'est jamais stockée sur le serveur de stockage	Compromission de l'espace de stockage ou du fournisseur
Secrets	GitHub Secrets pour la chaîne de livraison, fichier	Fuite d'identifiants par le dépôt ou les

Niveau	Mise en œuvre	Menace couverte
	. env en droits 600 sur le serveur ; rotation annuelle et rotation immédiate en cas de suspicion	journaux

Un arbitrage doit être explicité : les émotions elles-mêmes (identifiant, intensité, date) ne sont **pas** chiffrées au niveau colonne, contrairement aux commentaires. Les chiffrer interdirait tout calcul statistique en base, donc la fonctionnalité de rapport par période, cœur du module. Le compromis retenu protège l'élément le plus révélateur sans sacrifier le service rendu.

4.5 Données personnelles et conformité RGPD

4.5.1 Nature des traitements

CESIZen traite des données concernant la santé au sens de l'article 9 du RGPD. Le traitement repose sur le **consentement explicite** de la personne (article 9.2.a), recueilli à la création du compte par une action positive distincte de l'acceptation des conditions d'utilisation. Une **analyse d'impact relative à la protection des données (AIPD)** est obligatoire et a été réalisée : le traitement combine deux critères de la liste CNIL — données sensibles et collecte à grande échelle.

4.5.2 Registre des traitements et durées de conservation

Donnée	Finalité	Base légale	Conservation
Adresse électronique	Identification, authentification, notifications de service	Exécution du service	Suppression du compte, ou 24 mois d'inactivité
Mot de passe (empreinte Argon2)	Authentification	Exécution du service	Idem compte
Prénom (facultatif)	Personnalisation de l'interface	Consentement	Idem compte
Entrées du journal d'émotions	Suivi émotionnel et statistiques personnelles	**Consentement explicite** (art. 9.2.a)	Idem compte ; suppression unitaire possible à tout moment
Articles consultés et favoris	Personnalisation des contenus	Consentement	12 mois glissants
Journaux de connexion (IP, horodatage)	Sécurité, détection d'intrusion	Intérêt légitime	**6 mois** (recommandation CNIL)
Journaux techniques applicatifs	Diagnostic d'anomalies	Intérêt légitime	12 mois
Sauvegardes	Continuité de service	Intérêt légitime	30 jours glissants

Les durées sont appliquées par une tâche planifiée (`manage.py purger_donnees`), exécutée quotidiennement et journalisée. Les comptes inactifs depuis 21 mois reçoivent une relance avant suppression à 24 mois : une suppression automatique sans avertissement priverait l'utilisateur d'un historique auquel il tient, sans nécessité.

4.5.3 Mise en œuvre des principes du règlement

Principe	Traduction concrète dans CESIZen
Minimisation	Ni nom, ni date de naissance, ni sexe, ni géolocalisation ne sont collectés. Le prénom est facultatif. L'inscription se limite à une adresse électronique et un mot de passe : rien de ce qui n'est pas nécessaire au service n'est demandé.
Protection dès la conception	Identifiants UUID (aucune énumération possible des utilisateurs), chiffrement des champs libres, cloisonnement réseau, contrôle « impact sur les données personnelles » obligatoire dans chaque *pull request*.
Droit d'accès et à la portabilité	Export complet au format JSON depuis l'espace personnel, disponible immédiatement ; aucune démarche auprès du prestataire n'est nécessaire.
Droit à l'effacement	Suppression du compte depuis l'application : effacement immédiat en base, purge des sauvegardes sous 30 jours (délai documenté dans la politique de confidentialité).
Droit de rectification	Modification directe du profil et des entrées du journal.
Sécurité (art. 32)	Ensemble des mesures des § 4.3 et 4.4.
Localisation des données	Hébergement OVHcloud à Gravelines (France) ; supervision Sentry sur instance européenne ; **aucun transfert hors Union européenne**. Ni Google Analytics, ni service de notification tiers hors UE.

Principe	Traduction concrète dans CESIZen
Sous-traitance (art. 28)	OVHcloud est le seul sous-traitant au sens du règlement ; le contrat de sous-traitance et ses garanties sont annexés au registre.
Violation de données (art. 33-34)	Procédure décrite au § 4.6 : notification à la CNIL sous 72 heures, information des personnes concernées en cas de risque élevé.

4.6 Gestion de crise et escalade de l'information

Un plan de sécurisation qui s'arrête à la prévention est incomplet : il faut décrire ce qui se passe le jour où les mesures sont mises en défaut. Trois niveaux d'escalade sont définis, chacun mobilisant des acteurs et déclenchant des obligations différentes.

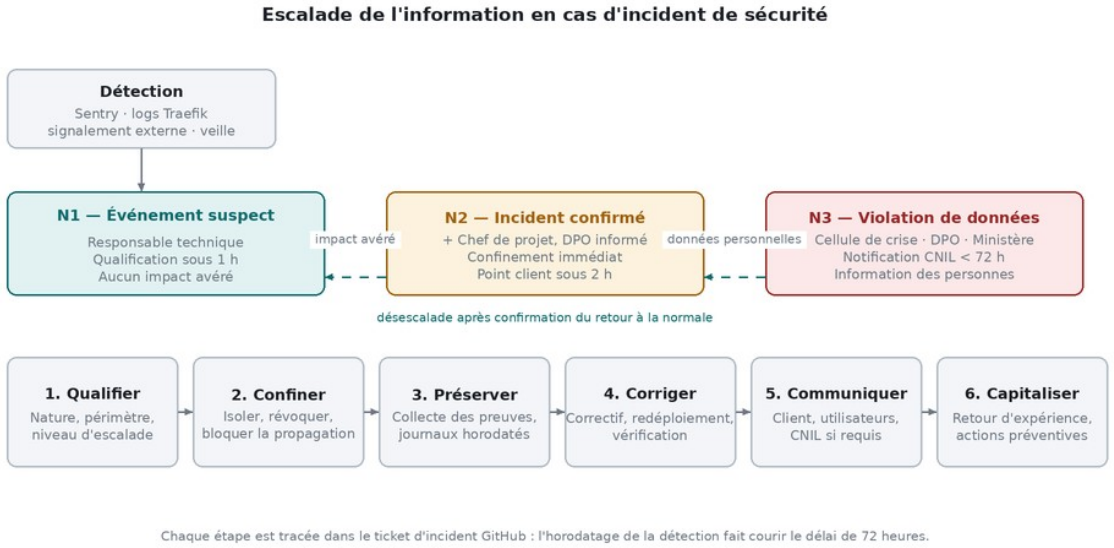


Figure 3 — Niveaux d'escalade et étapes de traitement d'un incident de sécurité.

Niveau	Déclencheur	Acteurs mobilisés	Actions et délais
N1 Événement suspect	Alerte de supervision, pic de tentatives d'authentification, vulnérabilité publiée	Responsable technique (prestataire)	Qualification sous 1 h ; ticket `[SECURITE]` ouvert ; si aucun impact avéré, surveillance renforcée pendant 72 h
N2 Incident confirmé	Intrusion avérée, indisponibilité d'origine malveillante, exposition d'un secret	+ Chef de projet, délégué à la protection des données **informé**, référent Ministère	Confinement immédiat ; point au commanditaire sous 2 h puis toutes les 4 h ; collecte et conservation des preuves
N3 Violation de données	Accès non autorisé à des données personnelles avéré ou probable	**Cellule de crise** : prestataire, chef de projet, délégué à la protection des données, direction du Ministère, communication	Réunion sous 2 h ; **notification à la CNIL sous 72 h** ; information des personnes concernées si risque élevé (art. 34) ; communication publique préparée par le Ministère

Principes de fonctionnement de la chaîne d'escalade :

- **L'escalade est un devoir, pas une option.** En cas de doute sur la qualification, le niveau supérieur est retenu. Sous-évaluer un incident coûte toujours plus cher que de mobiliser inutilement.
- **Le délai de 72 heures court dès la détection**, pas dès la compréhension complète de l'incident. La notification à la CNIL peut être faite par étapes : une notification initiale incomplète, complétée ensuite, est préférable à une notification tardive.
- **La communication publique appartient au Ministère.** Le prestataire fournit les éléments factuels et techniques ; il ne s'exprime pas publiquement.
- **La désescalade est formalisée.** Le retour au niveau inférieur, puis à la normale, est prononcé explicitement et tracé dans le ticket, après confirmation que la cause a été traitée et non seulement le symptôme.

- **Chaque incident N2 ou N3 donne lieu à un retour d'expérience** sous quinze jours, produisant des tickets d'actions préventives : un incident qui ne modifie ni le code, ni la supervision, ni les procédures est un incident dont on n'a rien appris.

4.7 Bonnes pratiques de développement

Les mesures qui précèdent ne tiennent que si les pratiques quotidiennes de développement les soutiennent. Celles retenues sont **outillées** : chacune est vérifiée automatiquement ou imposée par la configuration du dépôt, plutôt que laissée à la discipline individuelle.

Domaine	Pratique	Contrôle
Structure du code	Architecture MVC respectée : logique métier dans les modèles et services, aucune requête en base dans les vues ; découpage en applications Django par module fonctionnel	Revue de code, `CODEOWNERS`
Style et lisibilité	Formatage automatique `black`, analyse `ruff`, annotations de type sur les fonctions publiques	Bloquant en intégration continue
Configuration	Méthode des douze facteurs : aucune valeur d'environnement dans le code, tout par variables ; réglages séparés développement / test / production	`manage.py check --deploy` bloquant
Gestion des secrets	Aucun secret dans le dépôt, `.env` ignoré, fichier `.env.example` documenté	Détection de secrets, `gitleaks`
Tests	Test unitaire pour toute règle de gestion ; test d'autorisation pour tout point d'accès ; test de non-régression pour toute anomalie corrigée	Couverture minimale de 70 % bloquante
Revue de code	Aucune fusion directe : *pull request* obligatoire avec liste de contrôle, y compris pour le prestataire unique	Protection de branche
Migrations	Migrations réversibles ; séparation d'un changement de schéma et d'une migration de données ; jamais de suppression de colonne dans la même version que l'arrêt de son usage	Revue de code, sauvegarde préalable
Journalisation	Journaux structurés, **jamais de donnée personnelle ni de secret** ; horodatage UTC ; niveaux cohérents	Filtre de journalisation Django, revue
Dépendances	Versions figées (`requirements.txt` verrouillé) ; toute nouvelle dépendance justifiée en revue (maintenance active, licence, taille)	Dependabot, `pip-audit`
Documentation	Documentation versionnée avec le code ; décisions d'architecture consignées ; guide d'installation testé à chaque reconstruction de serveur	Revue de code

> **Le principe directeur : rendre la mauvaise pratique difficile**
>
> La différence entre une bonne pratique affichée et une bonne pratique appliquée tient à ce qui se passe lorsqu'on l'oublie. Ici, oublier un test fait échouer l'intégration continue ; oublier un secret dans un commit déclenche un blocage à la poussée ; contourner la revue est empêché par la protection de branche. Cette approche est ce qui rend le dispositif tenable pour un prestataire unique, qui ne dispose ni d'un pair pour le relire, ni d'une équipe sécurité pour le contrôler.

5. Conclusion

Le dispositif décrit dans ce dossier repose sur une idée simple : avec un prestataire unique et un budget d'infrastructure inférieur à 600 € par an, la seule façon de tenir les exigences d'un service public traitant des données de santé est de **transformer les intentions en automatismes**.

C'est ce qui relie les trois volets. Le déploiement est automatisé : une mise en production ne dépend plus d'une procédure manuelle mémorisée et le retour arrière est toujours possible. La maintenance automatise la détection — tickets créés par les processus, veille hebdomadaire, montées de version validées par les tests — de sorte que le portefeuille reflète l'état réel du produit. La sécurisation automatise le contrôle : vulnérabilité connue, secret exposé ou configuration dangereuse sont arrêtés par la chaîne d'intégration avant d'atteindre la production.

L'analyse EBIOS ramène les seize risques identifiés d'une situation initiale comportant trois risques critiques et huit risques élevés à un profil résiduel sans risque élevé. Ce résultat vaut à la date de rédaction et pour le périmètre décrit : c'est précisément pourquoi la veille et la revue mensuelle des indicateurs font partie intégrante du dispositif, et non de son ornement.

Annexes — Éléments configurés et éléments de preuve

Les fichiers de configuration décrits dans ce dossier sont versionnés dans le dépôt du projet et présentés lors de la soutenance. Les copies d'écran correspondantes sont jointes en annexe.

Annexe	Élément	Emplacement dans le dépôt
A	Image de l'API et composition des environnements de développement et de production	`deploy/Dockerfile`, `deploy/docker-compose.yml`, `deploy/docker-compose.prod.yml`
B	En-têtes de sécurité HTTP et limitation de débit	`deploy/traefik/dynamic/middlewares.yml`
C	Sauvegarde chiffrée et procédure de restauration	`deploy/scripts/`
D	Chaîne d'intégration continue (qualité, tests, sécurité, mobile)	`.github/workflows/ci.yml`
E	Déploiement automatisé en recette	`.github/workflows/deploy-staging.yml`
F	Déploiement en production avec approbation et retour arrière	`.github/workflows/deploy-production.yml`
G	Veille de sécurité hebdomadaire automatisée	`.github/workflows/securite-planifiee.yml`
H	Mise à jour automatisée des dépendances	`.github/dependabot.yml`
I	Formulaires de tickets : anomalie, évolution, incident de sécurité	`.github/ISSUE_TEMPLATE/`
J	Référentiel d'étiquettes et modèle de *pull request*	`.github/labels.yml`, `.github/PULL_REQUEST_TEMPLATE.md`
K	Guide d'installation et d'exploitation	`deploy/README.md`
L	Guide de démonstration et déroulé de soutenance	`docs/DEMO.md`

***Copies d'écran à joindre** : exécution réussie de la chaîne d'intégration continue avec le détail des quatre travaux ; rapport de couverture de tests ; onglet Security du dépôt ; historique des déploiements avec l'étape d'approbation en production ; tableau de pilotage GitHub Projects ; ticket d'anomalie renseigné avec sa sévérité ; ticket de veille hebdomadaire généré automatiquement ; pull request Dependabot validée par l'intégration continue.*